Week 2 Assignment: Insurance Management System (WebAPI)

Develop a Controller-Based ASP.NET Core Web API for an Insurance Company. This is an intermediate-level assignment which teaches you - how to build a real ASP.NET Core Web API without introducing Entity Framework, SQL Server, Repository Pattern, Unit of Work, or any architecture pattern. Instead, it focuses on the fundamentals that every ASP.NET Core developer should master. Later on, you can extend it to Database and Architecture based real life application.

Scenario

XYZ Insurance provides - Health Insurance, Motor Insurance, Life Insurance. Customers purchase policies. Each policy belongs to one customer. The company also records claims against policies.

Project Structure

- InsuranceApi
  - Controllers
    - CustomerController.cs
    - PolicyController.cs
    - ClaimController.cs
  - Models
    - Customer.cs
    - Policy.cs
    - Claim.cs
  - DTOs
    - Customer
      - CustomerCreateDto.cs
      - CustomerUpdateDto.cs
      - CustomerResponseDto.cs
    - Policy
      - PolicyCreateDto.cs
      - PolicyUpdateDto.cs
      - PolicyResponseDto.cs
    - Claim
      - ClaimCreateDto.cs
      - ClaimUpdateDto.cs
      - ClaimResponseDto.cs
  - Services
    - Interfaces
      - ICustomerService.cs
      - IPolicyService.cs
      - IClaimService.cs
    - Service Classes
      - CustomerService.cs
      - PolicyService.cs
      - ClaimService.cs

Models

- **Customer Model**
  - CustomerId, FullName, Email, Phone, DOB, City, State and IsActive
- **Policy Model**
  - PolicyId, PolicyNumber, PolicyType, Premium, CoverageAmount, StartDate, EndDate, Status, CustomerId
- **Claim Model**
  - ClaimId, ClaimNumber, ClaimAmount, ClaimDate, Status, PolicyId

Use Data Annotations wherever required in Models

Required, StringLength, EmailAddress, Phone, Range, RegularExpression

Static Data

Maintain data inside services.

Example

```
private static List<Customer> customers = new();
private static List<Policy> policies = new();
private static List<Claim> claims = new();
```

Endpoints

CustomerController		
Request	**EndPoint**	**Purpose**
GET	**/api/customer**	**Return all customers**
GET	**/api/customer/{id}**	**Return customer by Id**
POST	**/api/customer**	**Create customer**
PUT	**/api/customer/{id}**	**Update customer**
PATCH	**/api/customer/{id}/status**	**Enable or Disable customer**
DELETE	**/api/customer/{id}**	**Delete customer, cannot delete if customer owns policies.**

PolicyController		
Request	**EndPoint**	**Purpose**
GET	**/api/policy**	**Return all policy**
GET	**/api/policy/{id}**	**Return policies by Id**
GET	**/api/policy/customer/{customerId}**	**Return policies of a customer.**
POST	**/api/policy**	**Create customer** **Rules:** ○ **Customer must exist.** ○ **Policy Number should be unique.** ○ **Premium > 0** ○ **Coverage > Premium**
PUT	**/api/policy/{id}**	**Update Policy**
PATCH	**/api/policy/{id}/cancel**	**Cancel policy**
DELETE	**/api/policy/{id}**	**Cannot delete if claims exist.**

ClaimController		
Request	**EndPoint**	**Purpose**
GET	**/api/claim**	**Return all claims**
GET	**/api/ claim /{id}**	**Return claims by Id**
GET	**/api/claim/policy/{policyId}**	**Return claims of a policy holder**
POST	**/api/claim**	**Create Claim** **Rules:** ○ **Policy should exist.** ○ **Policy should be Active.** ○ **Claim Amount** ○ **cannot exceed Coverage Amount.**
PATCH	**/api/claim/{id}/approve**	**Approve Claim**
PATCH	**/api/claim/{id}/reject**	**Reject Claim**
DELETE	**/api/claim/{id}**	**Delete Claim**

Service Layer

Each service should perform

- Validation
- CRUD
- Business Logic
- List manipulation

Controller should contain almost no business logic.

Expected HTTP Status Codes

- 200 OK
- 201 Created
- 204 No Content
- 400 Bad Request
- 404 Not Found
- 409 Conflict

Reference Solution (High-Level)

A complete reference implementation should include:

- **3 Models:** Customer, Policy, Claim.
- **9 DTOs:** Create, Update, and Response DTOs for each entity.
- **3 Service Interfaces:** ICustomerService, IPolicyService, IClaimService.
- **3 Service Implementations:** Registered as singletons and managing static List<T> collections with seeded data.
- **4 Controllers:** CustomerController, PolicyController, ClaimController, and DashboardController.

- Controllers that call only the service layer and return appropriate HTTP status codes (Ok, CreatedAtAction, NoContent, BadRequest, NotFound, Conflict).
- Business validations implemented exclusively in the service layer.
- DTO-to-Model and Model-to-DTO mapping performed manually (no AutoMapper).
- Full testing through Swagger, covering all CRUD operations and business rule scenarios listed above.

Swagger Testing Tasks

- Students should verify

  ✓ Duplicate Email
  ✓ Duplicate Phone
  ✓ Invalid Premium
  ✓ Invalid Coverage
  ✓ Customer Not Found
  ✓ Policy Not Found
  ✓ Invalid Claim
  ✓ Delete Customer having Policies
  ✓ Delete Policy having Claims
  ✓ Approve Claim
  ✓ Reject Claim
  ✓ Update Customer
  ✓ Update Policy
  ✓ Update Claim

This assignment gives you hands-on experience with nearly every core feature of controller-based ASP.NET Core Web APIs before introducing persistence frameworks or architectural patterns, making it an ideal bridge from beginner to enterprise-style API development.

.NET: The Power to Do More